

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 805 «Математическая кибернетика»

**Лабораторные работы
по курсу «Численные методы»**



Москва, 2026

Содержание

1	Лабораторная работа 1. Прямые и итерационные методы решения СЛАУ	3
1.1	Задание 1.1	3
1.1.1	Главные части кода	3
1.1.2	Результаты работы программы	4
1.2	Задание 1.2	5
1.2.1	Главные части кода	5
1.2.2	Результаты работы программы	6
1.3	Задание 1.3	6
1.3.1	Главные части кода	6
1.3.2	Результаты работы программы	7
1.4	Задание 1.4	7
1.4.1	Главные части кода	8
1.4.2	Результаты работы программы	9
1.5	Задание 1.5	9
1.5.1	Главные части кода	9
1.5.2	Результаты работы программы	12
2	Лабораторная работа 2. Методы решения нелинейных уравнений и систем	13
2.1	Задание 2.1	13
2.1.1	Главные части кода	13
2.1.2	Результаты работы программы	14
2.2	Задание 2.2	15
2.2.1	Главные части кода	15
2.2.2	Результаты работы программы	17
3	Лабораторная работа 3. Интерполяция и приближение функций	18
3.1	Задание 3.1	18
3.1.1	Главные части кода	18
3.1.2	Результаты работы программы	19
3.2	Задание 3.2	19
3.2.1	Главные части кода	19
3.2.2	Результаты работы программы	20
3.3	Задание 3.3	20
3.3.1	Главные части кода	21
3.3.2	Результаты работы программы	22
3.4	Задание 3.4	22
3.4.1	Главные части кода	22
3.4.2	Результаты работы программы	24
3.5	Задание 3.5	24
3.5.1	Главные части кода	24
3.5.2	Результаты работы программы	25
4	Лабораторная работа 4. Численное решение дифференциальных уравнений	26
4.1	Задание 4.1	26
4.1.1	Главные части кода	26
4.1.2	Результаты работы программы	27
4.2	Задание 4.2	28

4.2.1	Главные части кода	28
4.2.2	Результаты работы программы	30

1 Лабораторная работа 1. Прямые и итерационные методы решения СЛАУ

1.1 Задание 1.1

Постановка задачи: Реализовать алгоритм LU - разложения матриц (с выбором главного элемента) в виде программы. Используя разработанное программное обеспечение, решить систему линейных алгебраических уравнений (СЛАУ). Для матрицы СЛАУ вычислить определитель и обратную матрицу.

$$17. \begin{cases} 8 \cdot x_1 + 8 \cdot x_2 - 5 \cdot x_3 - 8 \cdot x_4 = 13 \\ 8 \cdot x_1 - 5 \cdot x_2 + 9 \cdot x_3 - 8 \cdot x_4 = 38 \\ 5 \cdot x_1 - 4 \cdot x_2 - 6 \cdot x_3 - 2 \cdot x_4 = 14 \\ 8 \cdot x_1 + 3 \cdot x_2 + 6 \cdot x_3 + 6 \cdot x_4 = -95 \end{cases}$$

1.1.1 Главные части кода

```
1 import copy
2
3 def lu_decomposition(matrix, eps=1e-12):
4     n = len(matrix)
5     U = copy.deepcopy(matrix)
6     L = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
7     P = list(range(n))
8     swaps = 0
9
10    for k in range(n):
11        max_row = k
12        for i in range(k + 1, n):
13            if abs(U[i][k]) > abs(U[max_row][k]):
14                max_row = i
15
16        if max_row != k:
17            U[k], U[max_row] = U[max_row], U[k]
18            P[k], P[max_row] = P[max_row], P[k]
19            for j in range(k):
20                L[k][j], L[max_row][j] = L[max_row][j], L[k][j]
21            swaps += 1
22
23        if abs(U[k][k]) < eps:
24            raise ValueError("Матрица вырождена или близка к этому")
25
26        for i in range(k + 1, n):
27            mu = U[i][k] / U[k][k]
28            L[i][k] = mu
29            for j in range(k, n):
30                U[i][j] -= mu * U[k][j]
31            U[i][k] = 0
32
33    sign = (-1) ** swaps
34    return L, U, P, sign
35
```

```

36
37 def solve_system(L, U, P, b):
38     n = len(L)
39     z = [0.0] * n
40     for i in range(n):
41         sum_val = 0
42         for j in range(i):
43             sum_val += L[i][j] * z[j]
44         z[i] = b[P[i]] - sum_val
45
46     x = [0.0] * n
47     for i in range(n - 1, -1, -1):
48         sum_val = 0
49         for j in range(i + 1, n):
50             sum_val += U[i][j] * x[j]
51         x[i] = (z[i] - sum_val) / U[i][i]
52     return x
53
54
55 def get_inverse(matrix):
56     n = len(matrix)
57     L, U, P, _ = lu_decomposition(matrix)
58     inv = [[0.0] * n for _ in range(n)]
59
60     for col in range(n):
61         e = [1.0 if row == col else 0.0 for row in range(n)]
62         res_col = solve_system(L, U, P, e)
63         for row in range(n):
64             inv[row][col] = res_col[row]
65     return inv

```

Листинг 1: Реализация LU-разложения

1.1.2 Результаты работы программы

```

--- Решение СЛАУ ---
x1 = -4.000000
x2 = -3.000000
x3 = -1.000000
x4 = -8.000000

Определитель: 20050.0000

--- Обратная матрица ---
    0.016459    0.011671    0.061945    0.058155
    0.061347   -0.029227   -0.078204    0.016758
   -0.014464    0.044289   -0.072618    0.015561
   -0.038155   -0.045237    0.029127    0.065187

```

Рис. 1: Результат выполнения задания 1.1

1.2 Задание 1.2

Постановка задачи: Реализовать метод прогонки в виде программы, задавая в качестве входных данных ненулевые элементы матрицы системы и вектор правых частей. Используя разработанное программное обеспечение, решить СЛАУ с трехдиагональной матрицей.

$$17. \begin{cases} -6 \cdot x_1 + 5 \cdot x_2 = 51 \\ -x_1 + 13 \cdot x_2 + 6 \cdot x_3 = 100 \\ -9 \cdot x_2 - 15 \cdot x_3 - 4 \cdot x_4 = -12 \\ -x_3 - 7 \cdot x_4 + x_5 = 47 \\ 9 \cdot x_4 - 18 \cdot x_5 = -90 \end{cases}$$

1.2.1 Главные части кода

```
1 def solve_progonka(a, b, c, d):
2     n = len(d)
3
4     # проверка устойчивости: |bi| >= |ai| + |ci|
5     for i in range(n):
6         if abs(b[i]) < abs(a[i]) + abs(c[i]):
7             print(f"внимание: в строке {i + 1} условие устойчивости не выполнено")
8
9     p = [0.0] * n
10    q = [0.0] * n
11    x = [0.0] * n
12
13    # прямой ход
14    p[0] = -c[0] / b[0]
15    q[0] = d[0] / b[0]
16
17    # x_i = P_i*x_{i+1} + Q_i
18    for i in range(1, n):
19        m = b[i] + a[i] * p[i - 1]
20        p[i] = -c[i] / m
21        q[i] = (d[i] - a[i] * q[i - 1]) / m
22
23    # обратный ход
24    x[n - 1] = q[n - 1]
25    for i in range(n - 2, -1, -1):
26        x[i] = p[i] * x[i + 1] + q[i]
27
28    return x
```

Листинг 2: Метод прогонки и проверка устойчивости

1.2.2 Результаты работы программы

```
x1 = -1.0000
x2 = 9.0000
x3 = -3.0000
x4 = -6.0000
x5 = 2.0000
```

Рис. 2: Результат выполнения задания 1.2

1.3 Задание 1.3

Постановка задачи: Реализовать метод простых итераций и метод Зейделя в виде программ, задавая в качестве входных данных матрицу системы, вектор правых частей и точность вычислений. Используя разработанное программное обеспечение, решить СЛАУ. Проанализировать количество итераций, необходимое для достижения заданной точности.

$$17. \begin{cases} -19 \cdot x_1 + 2 \cdot x_2 - x_3 - 8 \cdot x_4 = 38 \\ 2 \cdot x_1 + 14 \cdot x_2 - 4 \cdot x_4 = 20 \\ 6 \cdot x_1 - 5 \cdot x_2 - 20 \cdot x_3 - 6 \cdot x_4 = 52 \\ -6 \cdot x_1 + 4 \cdot x_2 - 2 \cdot x_3 + 15 \cdot x_4 = 43 \end{cases}$$

1.3.1 Главные части кода

```
1 def get_alpha_beta(A, b):
2     n = len(b)
3     alpha = [[0.0] * n for _ in range(n)]
4     beta = [0.0] * n
5     for i in range(n):
6         diag = A[i][i]
7         beta[i] = b[i] / diag
8         for j in range(n):
9             if i != j:
10                alpha[i][j] = -A[i][j] / diag
11     return alpha, beta
12
13 def get_norm_matrix(matrix):
14     return max(sum(abs(x) for x in row) for row in matrix)
15
16 def get_norm_vector(v):
17     return max(abs(x) for x in v)
18
19 def simple_iteration(alpha, beta, eps):
```

```

20     n = len(beta)
21     x = list(beta)
22     norm_a = get_norm_matrix(alpha)
23     k = 0
24     while True:
25         k += 1
26         x_new = [0.0] * n
27         for i in range(n):
28             s = sum(alpha[i][j] * x[j] for j in range(n))
29             x_new[i] = beta[i] + s
30         diff = [x_new[i] - x[i] for i in range(n)]
31         if (norm_a / (1 - norm_a)) * get_norm_vector(diff) < eps:
32             return x_new, k
33         x = x_new
34
35 def seidel(alpha, beta, eps):
36     n = len(beta)
37     x = list(beta)
38     norm_a = get_norm_matrix(alpha)
39     k = 0
40     while True:
41         k += 1
42         x_prev = list(x)
43         for i in range(n):
44             s = sum(alpha[i][j] * x[j] for j in range(n))
45             x[i] = beta[i] + s
46         diff = [x[i] - x_prev[i] for i in range(n)]
47         if (norm_a / (1 - norm_a)) * get_norm_vector(diff) < eps:
48             return x, k

```

Листинг 3: Итерационные методы решения СЛАУ

1.3.2 Результаты работы программы

```

MPI: [-2.0000059534344636, 2.00000359058673, -4.0000030619859945, 1.0000017894555353], Steps: 22
Seidel: [-2.000003763960571, 2.0000023634751853, -4.000003637111827, 0.9999973792074788], Steps: 15

```

Рис. 3: Результат выполнения задания 1.3

1.4 Задание 1.4

Постановка задачи: Реализовать метод вращений в виде программы, задавая в качестве входных данных матрицу и точность вычислений. Используя разработанное программное обеспечение, найти собственные значения и собственные векторы симметрических матриц. Проанализировать зависимость погрешности вычислений от числа итераций.

$$17. \begin{pmatrix} 5 & -3 & -4 \\ -3 & -3 & 4 \\ -4 & 4 & 0 \end{pmatrix}$$

1.4.1 Главные части кода

```
1 import math
2
3 def get_max_off_diagonal(A):
4     n = len(A)
5     max_val = 0
6     p, q = 0, 1
7     for i in range(n):
8         for j in range(i + 1, n):
9             if abs(A[i][j]) > max_val:
10                 max_val = abs(A[i][j])
11                 p, q = i, j
12     return p, q, max_val
13
14
15 def matrix_multiply(A, B):
16     n = len(A)
17     C = [[0.0] * n for _ in range(n)]
18     for i in range(n):
19         for j in range(n):
20             for k in range(n):
21                 C[i][j] += A[i][k] * B[k][j]
22     return C
23
24
25 def transpose(A):
26     return [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]
27
28
29 def jacobi_rotations(matrix, eps):
30     n = len(matrix)
31     A = [row[:] for row in matrix]
32     V = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
33
34     while True:
35         p, q, max_val = get_max_off_diagonal(A)
36
37         off_diag_norm = 0
38         for i in range(n):
39             for j in range(i + 1, n):
40                 off_diag_norm += A[i][j] ** 2
41         off_diag_norm = math.sqrt(2 * off_diag_norm)
42
43         if off_diag_norm < eps:
44             break
45
46         if abs(A[p][p] - A[q][q]) < 1e-10:
47             phi = math.pi / 4
48         else:
49             phi = 0.5 * math.atan(2 * A[p][q] / (A[p][p] - A[q][q]))
50
51         c, s = math.cos(phi), math.sin(phi)
52
53         U = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
54         U[p][p], U[q][q] = c, c
55         U[p][q], U[q][p] = -s, s
56
57         A = matrix_multiply(transpose(U), matrix_multiply(A, U))
58         V = matrix_multiply(V, U)
```

```

59
60     eigenvalues = [A[i][i] for i in range(n)]
61     eigenvectors = transpose(V)
62     return eigenvalues, eigenvectors

```

Листинг 4: Метод вращений Якоби

1.4.2 Результаты работы программы

```

Eigenvalues:
[9.0066, -5.7765, -1.2301]
Eigenvectors:
[0.7803, -0.3643, -0.5083]
[0.0238, 0.8296, -0.5579]
[0.625, 0.4232, 0.656]

```

Рис. 4: Результат выполнения задания 1.4

1.5 Задание 1.5

Постановка задачи: Реализовать алгоритм QR – разложения матриц в виде программы. На его основе разработать программу, реализующую QR – алгоритм решения полной проблемы собственных значений произвольных матриц, задавая в качестве входных данных матрицу и точность вычислений. С использованием разработанного программного обеспечения найти собственные значения матрицы.

$$17. \begin{pmatrix} -6 & 1 & -4 \\ -6 & 8 & -2 \\ 2 & -9 & 5 \end{pmatrix}$$

1.5.1 Главные части кода

```

1 import math
2
3 def matrix_multiply(A, B):
4     n, m, k = len(A), len(A[0]), len(B[0])
5     C = [[0.0] * k for _ in range(n)]
6     for i in range(n):
7         for j in range(k):
8             for l in range(m):
9                 C[i][j] += A[i][l] * B[l][j]

```

```

10     return C
11
12
13 def transpose(A):
14     return [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]
15
16
17 def get_norm(v):
18     return math.sqrt(sum(x ** 2 for x in v))
19
20
21 def qr_decomposition(A):
22     n = len(A)
23     R = [row[:] for row in A]
24     Q = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
25
26     for k in range(n - 1):
27         b = [R[i][k] for i in range(k, n)]
28         norm_b = get_norm(b)
29
30         v = [0.0] * len(b)
31         sign = 1.0 if b[0] >= 0 else -1.0
32         v[0] = b[0] + sign * norm_b
33         for i in range(1, len(b)):
34             v[i] = b[i]
35
36         norm_v = get_norm(v)
37         if norm_v == 0:
38             continue
39
40         H_small = [[1.0 if i == j else 0.0 for j in range(len(b))] for i
41 in range(len(b))]
42         for i in range(len(b)):
43             for j in range(len(b)):
44                 H_small[i][j] -= 2.0 * v[i] * v[j] / (norm_v ** 2)
45
46         H = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
47         for i in range(len(b)):
48             for j in range(len(b)):
49                 H[i + k][j + k] = H_small[i][j]
50
51         R = matrix_multiply(H, R)
52         Q = matrix_multiply(Q, H)
53
54     return Q, R
55
56 def extract_eigenvalues(A, eps):
57     n = len(A)
58     eigenvalues = []
59     i = 0
60     while i < n:
61         lower_sum = math.sqrt(sum(A[row][i] ** 2 for row in range(i + 1, n
62 )))
63
64         if i == n - 1 or lower_sum < eps:
65             eigenvalues.append(round(A[i][i], 4))
66             i += 1
67         else:
68             a11, a12 = A[i][i], A[i][i + 1]

```

```

68         a21, a22 = A[i + 1][i], A[i + 1][i + 1]
69
70         trace = a11 + a22
71         det = a11 * a22 - a12 * a21
72         discriminant = trace ** 2 - 4 * det
73
74         if discriminant >= 0:
75             eigenvalues.append(round((trace + math.sqrt(discriminant))
/ 2, 4))
76             eigenvalues.append(round((trace - math.sqrt(discriminant))
/ 2, 4))
77         else:
78             real = round(trace / 2, 4)
79             imag = round(math.sqrt(-discriminant) / 2, 4)
80             eigenvalues.append(complex(real, imag))
81             eigenvalues.append(complex(real, -imag))
82         i += 2
83     return eigenvalues
84
85
86 def qr_algorithm(A, eps):
87     A_k = [row[:] for row in A]
88     prev_vals = []
89
90     for iteration in range(1, 1000):
91         Q, R = qr_decomposition(A_k)
92         A_k = matrix_multiply(R, Q)
93         curr_vals = extract_eigenvalues(A_k, eps)
94
95         if prev_vals:
96             diff = max(abs(c - p) for c, p in zip(curr_vals, prev_vals))
97             if diff < eps:
98                 return curr_vals
99         prev_vals = curr_vals
100     return prev_vals

```

Листинг 5: QR-алгоритм вычисления собственных значений

1.5.2 Результаты работы программы

```
1. Исходная матрица A:
[ -6.0000  1.0000 -4.0000 ]
[ -6.0000  8.0000 -2.0000 ]
[  2.0000 -9.0000  5.0000 ]

-----

2. QR-разложение матрицы A:
Матрица Q (ортогональная):
[ -0.6882  0.5311 -0.4942 ]
[ -0.6882 -0.2626  0.6763 ]
[  0.2294  0.8056  0.5462 ]

Матрица R (верхняя треугольная):
[  8.7178 -8.2590  5.2766 ]
[ -0.0000 -8.8198  2.4287 ]
[  0.0000  0.0000  3.3555 ]

-----

3. Проверка разложения (Q * R):
[ -6.0000  1.0000 -4.0000 ]
[ -6.0000  8.0000 -2.0000 ]
[  2.0000 -9.0000  5.0000 ]

-----

Сошлось за 24 итераций
4. QR-алгоритм для собственных значений (eps = 0.001):
Собственные значения lambda_i:
lambda_1 = 8.3214
lambda_2 = -6.2683
lambda_3 = 4.9469
```

Рис. 5: Результат выполнения задания 1.5

2 Лабораторная работа 2. Методы решения нелинейных уравнений и систем

2.1 Задание 2.1

Постановка задачи: Реализовать методы простой итерации и Ньютона решения нелинейных уравнений в виде программ, задавая в качестве входных данных точность вычислений. С использованием разработанного программного обеспечения найти положительный корень нелинейного уравнения (начальное приближение определить графически). Проанализировать зависимость погрешности вычислений от количества итераций.

$$17. \quad 4^x - 5x - 2 = 0.$$

2.1.1 Главные части кода

```
1 import numpy as np
2
3 def f(x):
4     return 4 ** x - 5 * x - 2
5
6
7 def df(x):
8     return 4 ** x * np.log(4) - 5
9
10
11 def phi(x):
12     return np.log(5 * x + 2) / np.log(4)
13
14
15 def newton_method(x0, eps, x_true):
16     history = []
17     x = x0
18     k = 0
19     while True:
20         fx = f(x)
21         dfx = df(x)
22         dx = -fx / dfx
23         error = abs(x - x_true)
24         history.append((k, x, error))
25
26         if abs(dx) < eps:
27             break
28         x = x + dx
29         k += 1
30     return history
31
32
33 def simple_iteration(x0, eps, x_true):
34     history = []
35     x = x0
36     k = 0
37     q = 0.52 % Константа сжатия
38     while True:
39         x_next = phi(x)
40         error = abs(x_next - x_true)
41         history.append((k, x, error))
```

```

42
43     if (q / (1 - q)) * abs(x_next - x) < eps:
44         history.append((k + 1, x_next, abs(x_next - x_true)))
45         break
46     x = x_next
47     k += 1
48 return history

```

Листинг 6: Методы Ньютона и простой итерации для одного уравнения

2.1.2 Результаты работы программы

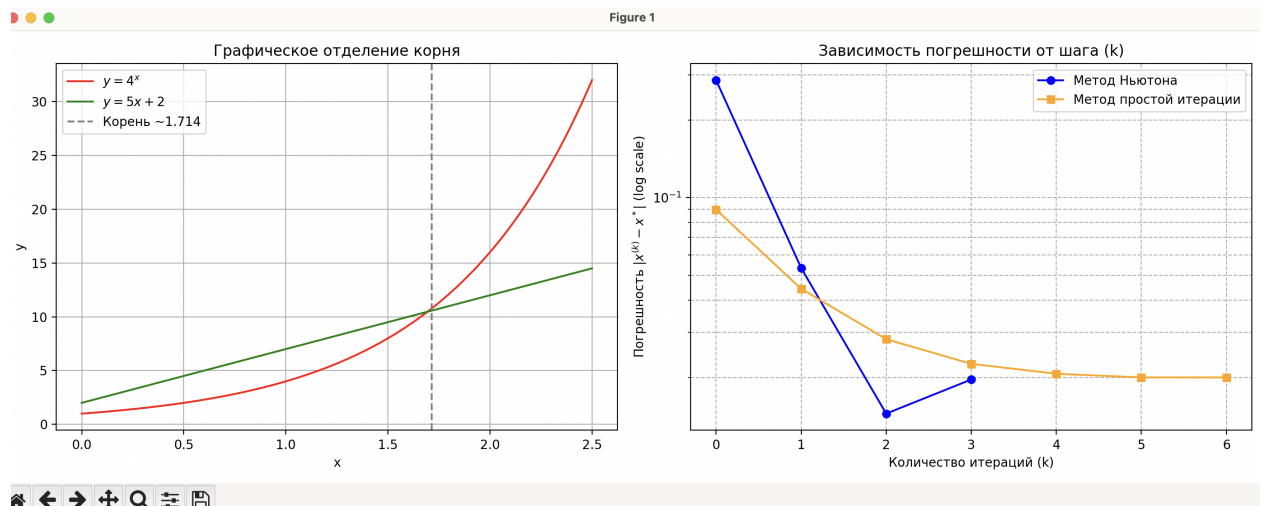


Рис. 6: Графическая локализация корня

--- МЕТОД НЬЮТОНА ---		
k	x^(k)	Погрешность
0	2.00000	0.28617
1	1.76718	0.05335
2	1.69934	0.01449
3	1.69417	0.01966

--- МЕТОД ПРОСТОЙ ИТЕРАЦИИ ---		
k	x^(k)	Погрешность
0	1.50000	0.08987
1	1.62396	0.04427
2	1.66956	0.02821
3	1.68562	0.02263
4	1.69120	0.02070
5	1.69313	0.02003
6	1.69380	0.02003

Рис. 7: Результаты и таблицы сходимости методов для нелинейного уравнения

2.2 Задание 2.2

Постановка задачи: Реализовать методы простых итераций и Ньютона решения систем нелинейных уравнений в виде программного кода, задавая в качестве входных данных точность вычислений. С использованием разработанного программного обеспечения решить систему нелинейных уравнений (при наличии нескольких решений найти то из них, в котором значения неизвестных являются положительными); начальное приближение определить графически. Проанализировать зависимость погрешности вычислений от количества итераций.

$$\begin{cases} ax_1 - \cos x_2 = 0, \\ ax_2 - e^{x_1} = 0. \end{cases}$$

a=3

2.2.1 Главные части кода

```
1 import numpy as np
2
```



```

3 def F(x):
4     return np.array([
5         3 * x[0] - np.cos(x[1]),
6         3 * x[1] - np.exp(x[0])
7     ])
8
9
10 def J(x):
11     return np.array([
12         [3, np.sin(x[1])],
13         [-np.exp(x[0]), 3]
14     ])
15
16
17 def Phi(x):
18     return np.array([
19         np.cos(x[1]) / 3,
20         np.exp(x[0]) / 3
21     ])
22
23
24 def newton_system(x0, eps, x_exact_calc=None):
25     history = []
26     x = x0.copy()
27     k = 0
28     while True:
29         fx = F(x)
30         jx = J(x)
31         dx = np.linalg.solve(jx, -fx)
32
33         if x_exact_calc is not None:
34             error = np.max(np.abs(x - x_exact_calc))
35         else:
36             error = np.max(np.abs(dx))
37
38         history.append((k, x.copy(), error))
39
40         if np.max(np.abs(dx)) < eps:
41             break
42         x += dx
43         k += 1
44     return history
45
46
47 def simple_iteration_system(x0, eps, x_exact_calc):
48     history = []
49     x = x0.copy()
50     k = 0
51     q = 0.55 % Константа сжатия
52     while True:
53         x_next = Phi(x)
54         error = np.max(np.abs(x_next - x_exact_calc))
55         history.append((k, x.copy(), error))
56
57         if (q / (1 - q)) * np.max(np.abs(x_next - x)) < eps:
58             history.append((k + 1, x_next.copy(), np.max(np.abs(x_next -
59                 x_exact_calc))))
60             break
61         x = x_next
62         k += 1

```

Листинг 7: Решение систем нелинейных уравнений

2.2.2 Результаты работы программы

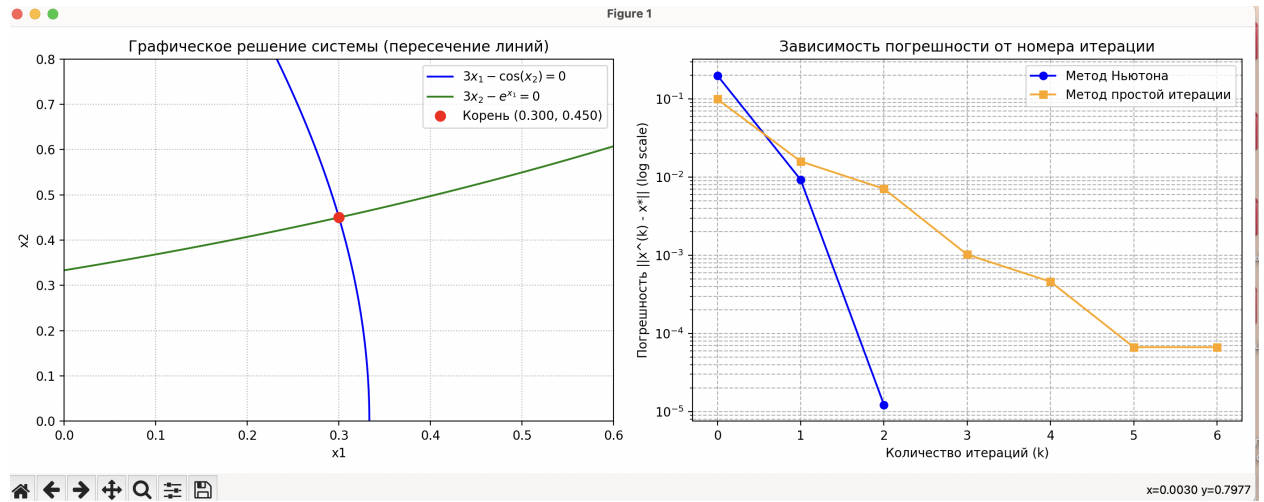


Рис. 8: Графическая локализация корня

```

--- МЕТОД НЬЮТОНА СИСТЕМ ---
k    x1      x2      Погрешность
0    0.50000  0.50000  0.19985
1    0.30200  0.44076  0.00926
2    0.30016  0.45002  0.00001

--- МЕТОД ПРОСТОЙ ИТЕРАЦИИ СИСТЕМ ---
k    x1      x2      Погрешность
0    0.50000  0.50000  0.09955
1    0.29253  0.54957  0.01590
2    0.28425  0.44660  0.00710
3    0.30064  0.44292  0.00102
4    0.30117  0.45024  0.00046
5    0.30011  0.45048  0.00007
6    0.30008  0.45000  0.00007

```

Рис. 9: Результат выполнения задания 2.2

3 Лабораторная работа 3. Интерполяция и приближение функций

3.1 Задание 3.1

Постановка задачи: Используя таблицу значений Y_i функции $y = f(x)$, вычисленных в точках $X_i, i = 0, \dots, 3$, построить интерполяционные многочлены Лагранжа и Ньютона, проходящие через точки $\{X_i, Y_i\}$. Вычислить значение погрешности интерполяции в точке X^* .

$$17. y = e^x + x, \quad \text{а) } X_i = -2, -1, 0, 1; \quad \text{б) } X_i = -2, -1, 0.2, 1; \quad X^* = -0.5.$$

3.1.1 Главные части кода

```
1 import numpy as np
2
3 def lagrange_poly(x, nodes, y_nodes):
4     res = 0
5     n = len(nodes)
6     for i in range(n):
7         p = 1
8         for j in range(n):
9             if i != j:
10                 p *= (x - nodes[j]) / (nodes[i] - nodes[j])
11         res += y_nodes[i] * p
12     return res
13
14
15 def divided_differences(nodes, y_nodes):
16     n = len(nodes)
17     coef = np.zeros((n, n))
18     coef[:, 0] = y_nodes
19
20     for j in range(1, n):
21         for i in range(n - j):
22             coef[i, j] = (coef[i + 1, j - 1] - coef[i, j - 1]) / (nodes[i + j] - nodes[i])
23
24     return coef[0, :]
25
26
27 def newton_poly(x, nodes, coef):
28     n = len(nodes)
29     res = coef[0]
30     p = 1
31     for i in range(1, n):
32         p *= (x - nodes[i - 1])
33         res += coef[i] * p
34     return res
```

Листинг 8: Интерполяция Лагранжа и Ньютона

3.1.2 Результаты работы программы

```
Истинное значение f(x*) в точке -0.5: 0.106531

=== Набор а) Фиксированные целые узлы ===
Используемые узлы (x_i): [-2. -1.  0.  1.]
Значения в узлах (y_i):  [-1.86466 -0.63212  1.      3.71828]
Коэффициенты Ньютона (c_i): [-1.86466  1.23254  0.19979  0.11443]
Метод      Значение в x*  Абс. Погрешность
Лагранж    0.091081    1.54e-02
Ньютон     0.091081    1.54e-02
-----

=== Набор б) Узел 0 смещен в точку 0.2 ===
Используемые узлы (x_i): [-2. -1.  0.2  1. ]
Значения в узлах (y_i):  [-1.86466 -0.63212  1.4214  3.71828]
Коэффициенты Ньютона (c_i): [-1.86466  1.23254  0.2176  0.12077]
Метод      Значение в x*  Абс. Погрешность
Лагранж    0.083949    2.26e-02
Ньютон     0.083949    2.26e-02
-----
```

Рис. 10: Результат сравнения полиномов Лагранжа и Ньютона

3.2 Задание 3.2

Постановка задачи: Построить кубический сплайн для функции, заданной в узлах интерполяции, предполагая, что сплайн имеет нулевую кривизну при $x = x_0$ и $x = x_4$. Вычислить значение функции в точке $x = X^*$.

17. $X^* = -0.5$

i	0	1	2	3	4
x_i	-2.0	-1.0	0.0	1.0	2.0
f_i	-1.8647	-0.63212	1.0	3.7183	9.3891

3.2.1 Главные части кода

```
1 import numpy as np
2
3 def build_cubic_spline(x, f):
4     n = len(x) - 1
5     h = np.diff(x)
6
7     # Формирование СЛАУ для коэффициентов c
8     A = np.zeros((n - 1, n - 1))
9     B = np.zeros(n - 1)
10
11     for i in range(1, n):
12         idx = i - 1
13         B[idx] = 3 * ((f[i + 1] - f[i]) / h[i] - (f[i] - f[i - 1]) / h[i - 1])
14         A[idx, idx] = 2 * (h[i - 1] + h[i])
15         if idx > 0:
16             A[idx, idx - 1] = h[i - 1]
```

```

17         if idx < n - 2:
18             A[idx, idx + 1] = h[i]
19
20     # Решение системы и сборка полного вектора с краевыми условиями
21     c_internal = np.linalg.solve(A, B)
22     c = np.zeros(n + 1)
23     c[1:n] = c_internal
24
25     # Вычисление коэффициентов a, b, d
26     a = np.zeros(n)
27     b = np.zeros(n)
28     d = np.zeros(n)
29
30     for i in range(n):
31         a[i] = f[i]
32         if i < n - 1:
33             b[i] = (f[i + 1] - f[i]) / h[i] - (1 / 3) * h[i] * (c[i + 2] +
34             2 * c[i + 1])
35             d[i] = (c[i + 2] - c[i + 1]) / (3 * h[i])
36         else:
37             b[i] = (f[i + 1] - f[i]) / h[i] - (2 / 3) * h[i] * c[i + 1]
38             d[i] = -c[i + 1] / (3 * h[i])
39
40     return a, b, c, d
41
42 def evaluate_spline(X_star, x, a, b, c, d):
43     n = len(x) - 1
44     for i in range(n):
45         if x[i] <= X_star <= x[i + 1]:
46             dx = X_star - x[i]
47             S_X = (a[i] + b[i] * dx + (c[i + 1] / 2) * (dx ** 2) + (d[i] /
48             6) * (dx ** 3))
49             return S_X, i
50     return None, -1

```

Листинг 9: Построение кубического сплайна

3.2.2 Результаты работы программы

Интервал	a_i	b_i	c_i (S_i)	d_i
1	-1.86470	0.99736	0.24648	-0.01125
2	-0.63212	0.76991	0.21272	0.64949
3	1.00000	1.27750	2.16120	-0.72040
4	3.71830	5.67080	0.00000	-0.00000

Точка X* = -0.5 принадлежит интервалу №2 [-1.0; 0.0]
Значение функции в точке X*: S(X*) = -0.207044

Рис. 11: Коэффициенты сплайна и значение в точке

3.3 Задание 3.3

Постановка задачи: Для таблично заданной функции путем решения нормальной системы МНК найти приближающие многочлены а) 1-ой и б) 2-ой степени. Для каж-

дого из приближающих многочленов вычислить сумму квадратов ошибок. Построить графики приближаемой функции и приближающих многочленов.

17.

i	0	1	2	3	4	5
x_i	-3.0	-2.0	-1.0	0.0	1.0	2.0
y_i	-2.9502	-1.8647	-0.63212	1.0	3.7183	9.3891

3.3.1 Главные части кода

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.array([-3.0, -2.0, -1.0, 0.0, 1.0, 2.0])
5 y = np.array([-2.9502, -1.8647, -0.63212, 1.0, 3.7183, 9.3891])
6 N = len(x)
7
8 print("--- Расчеты МНК ---")
9
10 A1 = np.array([
11     [N, np.sum(x)],
12     [np.sum(x), np.sum(x**2)]
13 ])
14 B1 = np.array([np.sum(y), np.sum(x * y)])
15
16 a0, a1 = np.linalg.solve(A1, B1)
17 print(f"\n1-я степень: F1(x) = {a0:.4f} + {a1:.4f}*x")
18
19 f1_vals = a0 + a1 * x
20 Phi1 = np.sum((f1_vals - y) ** 2)
21 print(f"Сумма квадратов ошибок (Phi1): {Phi1:.4f}")
22
23 A2 = np.array([
24     [N, np.sum(x), np.sum(x**2)],
25     [np.sum(x), np.sum(x**2), np.sum(x**3)],
26     [np.sum(x**2), np.sum(x**3), np.sum(x**4)]
27 ])
28 B2 = np.array([np.sum(y), np.sum(x * y), np.sum(x**2 * y)])
29
30 a0_2, a1_2, a2_2 = np.linalg.solve(A2, B2)
31 print(f"\n2-я степень: F2(x) = {a0_2:.4f} + {a1_2:.4f}*x + {a2_2:.4f}*x^2")
32
33 f2_vals = a0_2 + a1_2 * x + a2_2 * x**2
34 Phi2 = np.sum((f2_vals - y) ** 2)
35 print(f"Сумма квадратов ошибок (Phi2): {Phi2:.4f}")

```

Листинг 10: Аппроксимация по МНК

3.3.2 Результаты работы программы

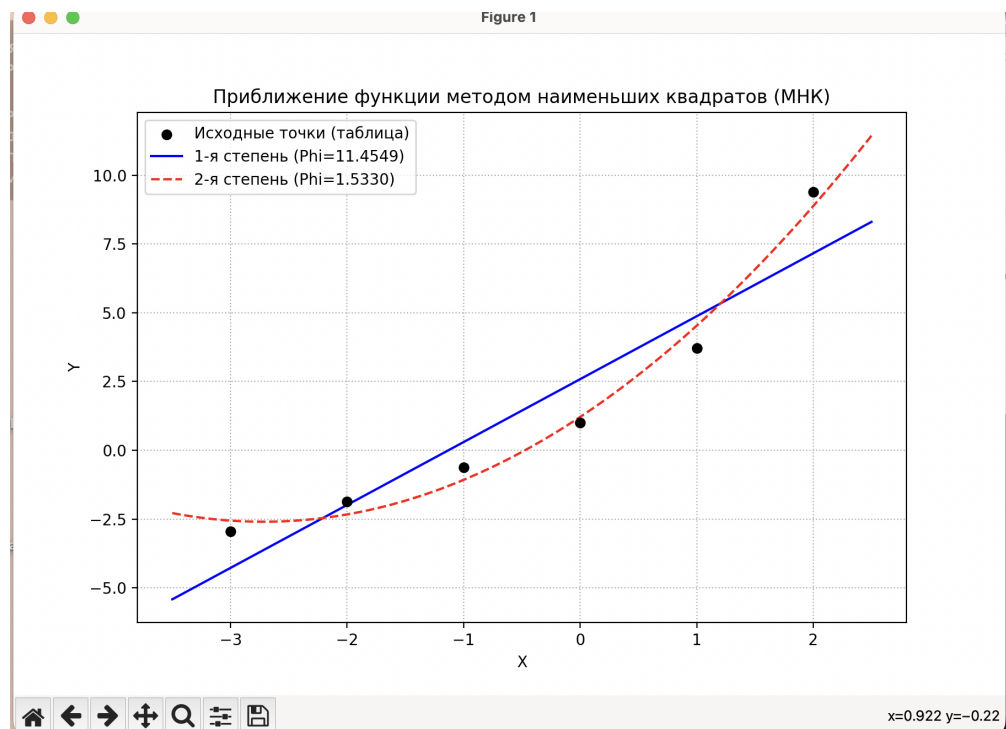


Рис. 12: График аппроксимации данных многочленами МНК

```
--- Расчеты МНК ---  
  
1-я степень: F1(x) = 2.5874 + 2.2879*x  
Сумма квадратов ошибок (Phi1): 11.4549  
  
2-я степень: F2(x) = 1.2126 + 2.8035*x + 0.5155*x^2  
Сумма квадратов ошибок (Phi2): 1.5330
```

Рис. 13: Результаты расчетов коэффициентов МНК

3.4 Задание 3.4

Постановка задачи: Вычислить первую и вторую производную от таблично заданной функции $y_i = f(x_i)$, $i = 0, 1, 2, 3, 4$ в точке $x = X^*$

17. $X^* = 0.2$

$\dot{x}$	0	1	2	3	4
x_i	-0.2	0.0	0.2	0.4	0.6
y_i	-0.40136	0.0	0.40136	0.81152	1.2435

3.4.1 Главные части кода

```

1 def main():
2     x = [-0.2, 0.0, 0.2, 0.4, 0.6]
3     y = [-0.40136, 0.0, 0.40136, 0.81152, 1.2435]
4     X_star = 0.2
5
6     try:
7         idx = x.index(X_star)
8     except ValueError:
9         print(f"Ошибка: Точка X* = {X_star} не найдена в таблице.")
10        return
11
12    print(f"== Численное дифференцирование для X* = {X_star} (индекс i = {idx}) ==")
13    print("-" * 60)
14
15    if idx > 0:
16        y_prime_left = (y[idx] - y[idx - 1]) / (x[idx] - x[idx - 1])
17        print(f"Левосторонняя производная y'({X_star}): {y_prime_left:.5f}")
18    else:
19        print("Левосторонняя производная: невозможно вычислить (нет левой точки)")
20
21    if idx < len(x) - 1:
22        y_prime_right = (y[idx + 1] - y[idx]) / (x[idx + 1] - x[idx])
23        print(f"Правосторонняя производная y'({X_star}): {y_prime_right:.5f}")
24    else:
25        print("Правосторонняя производная: невозможно вычислить (нет правой точки)")
26
27    print("-" * 60)
28
29    if idx > 0 and idx < len(x) - 1:
30        term1 = (y[idx] - y[idx - 1]) / (x[idx] - x[idx - 1])
31        diff_quotient1 = (y[idx + 1] - y[idx]) / (x[idx + 1] - x[idx])
32        diff_quotient2 = (y[idx] - y[idx - 1]) / (x[idx] - x[idx - 1])
33        term2 = (diff_quotient1 - diff_quotient2) / (x[idx + 1] - x[idx - 1])
34        multiplier = 2 * X_star - x[idx - 1] - x[idx + 1]
35
36        y_prime_second_order = term1 + term2 * multiplier
37        print(f"Первая производная (2-й порядок точности) y'({X_star}): {y_prime_second_order:.5f}")
38    else:
39        print("Второй порядок точности: недостаточно точек (нужны соседние слева и справа)")
40
41    print("-" * 60)
42
43    if idx > 0 and idx < len(x) - 1:
44        diff_quotient1 = (y[idx + 1] - y[idx]) / (x[idx + 1] - x[idx])
45        diff_quotient2 = (y[idx] - y[idx - 1]) / (x[idx] - x[idx - 1])
46
47        y_double_prime = 2 * (diff_quotient1 - diff_quotient2) / (x[idx + 1] - x[idx - 1])
48        print(f"Вторая производная y''({X_star}): {y_double_prime:.5f}")
49    else:
50        print("Вторая производная: невозможно вычислить (нужны соседние слева и справа)")

```


51 `print("-" * 60)`

Листинг 11: Расчет численных производных

3.4.2 Результаты работы программы

```
=== Численное дифференцирование для X* = 0.2 (индекс i = 2) ===
-----
Левосторонняя производная y'(0.2): 2.00680
Правосторонняя производная y'(0.2): 2.05080
-----
Первая производная (2-й порядок точности) y'(0.2): 2.00680
-----
Вторая производная y''(0.2): 0.22000
-----
```

Рис. 14: Значения производных 1-го и 2-го порядков точности

3.5 Задание 3.5

Постановка задачи: Вычислить определенный интеграл $F = \int_{X_0}^{X_1} y dx$, методами прямоугольников, трапеций, Симпсона с шагами h_1, h_2 . Оценить погрешность вычислений, используя Метод Рунге-Ромберга:

$$17. \quad y = \frac{1}{256 - x^4}, \quad X_0 = -2, \quad X_k = 2, \quad h_1 = 1.0, \quad h_2 = 0.5;$$

3.5.1 Главные части кода

```
1 def f(x):
2     return 1 / (256 - x ** 4)
3
4
5 def rectangles_method(x_0, x_k, h):
6     n = int((x_k - x_0) / h)
7     integral = 0
8     for i in range(n):
9         mid_x = x_0 + i * h + h / 2
10        integral += f(mid_x)
11    return integral * h
12
13
14 def trapezoids_method(x_0, x_k, h):
15     n = int((x_k - x_0) / h)
16     integral = (f(x_0) + f(x_k)) / 2
17     for i in range(1, n):
18         integral += f(x_0 + i * h)
19    return integral * h
20
21
22 def simpson_method(x_0, x_k, h):
23     n = int((x_k - x_0) / h)
```

```

24     integral = f(x_0) + f(x_k)
25
26     for i in range(1, n):
27         x_i = x_0 + i * h
28         if i % 2 != 0:
29             integral += 4 * f(x_i)
30         else:
31             integral += 2 * f(x_i)
32
33     return (h / 3) * integral
34
35 def runge_romberg(I_h1, I_h2, p):
36     """
37     Оценка погрешности по методу Рунге-Ромберга
38     I_h1 - интеграл с крупным шагом h1
39     I_h2 - интеграл с мелким шагом h2 (h2 = h1 / 2)
40     p     - порядок точности метода (p=2 для прямоугольников/трапеций, p=4
41     для Симпсона)
42     """
43     return abs(I_h2 - I_h1) / (2**p - 1)

```

Листинг 12: Численное интегрирование

3.5.2 Результаты работы программы

```

Интегрирование на отрезке [-2.0, 2.0]
=====
Метод прямоугольников:
  I (h1=1.0): 0.015785
  I (h2=0.5): 0.015816
  Погрешность Рунге-Ромберга: 1.051282e-05
-----
Метод трапеций:
  I (h1=1.0): 0.015916
  I (h2=0.5): 0.015850
  Погрешность Рунге-Ромберга: 2.192234e-05
-----
Метод Симпсона:
  I (h1=1.0): 0.015839
  I (h2=0.5): 0.015828
  Погрешность Рунге-Ромберга: 7.397476e-07
=====

```

Рис. 15: Результаты интегрирования с оценкой погрешностей

4 Лабораторная работа 4. Численное решение дифференциальных уравнений

4.1 Задание 4.1

Постановка задачи: Реализовать методы Эйлера, Рунге-Кутты и Адамса 4-го порядка в виде программы, задавая в качестве входных данных шаг сетки h . С использованием разработанного программного обеспечения решить задачу Коши для ОДУ 2-го порядка на указанном отрезке. Оценить погрешность численного решения с использованием метода Рунге – Ромберга и путем сравнения с точным решением.

17	$\begin{aligned} xy'' - (x+1)y' + y &= 0, \\ y(1) &= 2 + e, \\ y'(1) &= 1 + e, \\ x \in [1, 2], h &= 0.1 \end{aligned}$	$y = x + 1 + e^x$
----	---	-------------------

4.1.1 Главные части кода

```
1 import math
2
3 def exact_sol(x):
4     return x + 1 + math.exp(x)
5
6
7 def f_z(x, y, z):
8     return ((x + 1) * z - y) / x
9
10
11 def runge_kutta_step(x, y, z, h):
12     k1 = h * z
13     l1 = h * f_z(x, y, z)
14
15     k2 = h * (z + 0.5 * l1)
16     l2 = h * f_z(x + 0.5 * h, y + 0.5 * k1, z + 0.5 * l1)
17
18     k3 = h * (z + 0.5 * l2)
19     l3 = h * f_z(x + 0.5 * h, y + 0.5 * k2, z + 0.5 * l2)
20
21     k4 = h * (z + l3)
22     l4 = h * f_z(x + h, y + k3, z + l3)
23
24     dy = (k1 + 2 * k2 + 2 * k3 + k4) / 6
25     dz = (l1 + 2 * l2 + 2 * l3 + l4) / 6
26     return y + dy, z + dz
27
28
29 def solve_euler(x0, y0, z0, x_end, h):
30     N = int(round((x_end - x0) / h))
31     x_arr = [x0 + i * h for i in range(N + 1)]
32     y_arr = [0.0] * (N + 1)
33     z_arr = [0.0] * (N + 1)
34     y_arr[0], z_arr[0] = y0, z0
35
36     for k in range(N):
37         y_arr[k + 1] = y_arr[k] + h * z_arr[k]
38         z_arr[k + 1] = z_arr[k] + h * f_z(x_arr[k], y_arr[k], z_arr[k])
```

```

39     return x_arr, y_arr
40
41
42 def solve_rk4(x0, y0, z0, x_end, h):
43     N = int(round((x_end - x0) / h))
44     x_arr = [x0 + i * h for i in range(N + 1)]
45     y_arr = [0.0] * (N + 1)
46     z_arr = [0.0] * (N + 1)
47     y_arr[0], z_arr[0] = y0, z0
48
49     for k in range(N):
50         y_arr[k + 1], z_arr[k + 1] = runge_kutta_step(x_arr[k], y_arr[k],
51 z_arr[k], h)
52     return x_arr, y_arr
53
54 def solve_adams(x0, y0, z0, x_end, h):
55     N = int(round((x_end - x0) / h))
56     x_arr = [x0 + i * h for i in range(N + 1)]
57     y_arr = [0.0] * (N + 1)
58     z_arr = [0.0] * (N + 1)
59     y_arr[0], z_arr[0] = y0, z0
60
61     for k in range(3):
62         y_arr[k + 1], z_arr[k + 1] = runge_kutta_step(x_arr[k], y_arr[k],
63 z_arr[k], h)
64
65     for k in range(3, N):
66         f_y = lambda idx: z_arr[idx]
67         f_z_val = lambda idx: f_z(x_arr[idx], y_arr[idx], z_arr[idx])
68
69         y_arr[k + 1] = y_arr[k] + (h / 24) * (55 * f_y(k) - 59 * f_y(k -
70 1) + 37 * f_y(k - 2) - 9 * f_y(k - 3))
71         z_arr[k + 1] = z_arr[k] + (h / 24) * (
72             55 * f_z_val(k) - 59 * f_z_val(k - 1) + 37 * f_z_val(k
73 - 2) - 9 * f_z_val(k - 3))
74     return x_arr, y_arr

```

Листинг 13: Методы Эйлера, РК4 и Адамса

4.1.2 Результаты работы программы

x	Exact	Euler	RK4	Adams	Err Euler	Err RK4	Err Adams	Runge-Romb
1.0	4.718282	4.718282	4.718282	4.718282	0.00e+00	0.00e+00	0.00e+00	0.00e+00
1.1	5.104166	5.090110	5.104166	5.104166	1.41e-02	2.30e-07	2.30e-07	-
1.2	5.520117	5.489121	5.520116	5.520116	3.10e-02	5.09e-07	5.09e-07	4.66e-07
1.3	5.969297	5.918033	5.969296	5.969296	5.13e-02	8.44e-07	8.44e-07	-
1.4	6.455200	6.379836	6.455199	6.455187	7.54e-02	1.24e-06	1.25e-05	1.14e-06
1.5	6.981689	6.877820	6.981687	6.981661	1.04e-01	1.72e-06	2.79e-05	-
1.6	7.553032	7.415602	7.553030	7.552987	1.37e-01	2.28e-06	4.55e-05	2.09e-06
1.7	8.173947	7.997162	8.173944	8.173881	1.77e-01	2.94e-06	6.65e-05	-
1.8	8.849647	8.626879	8.849644	8.849556	2.23e-01	3.71e-06	9.16e-05	3.40e-06
1.9	9.585894	9.309566	9.585890	9.585773	2.76e-01	4.61e-06	1.21e-04	-
2.0	10.389056	10.050523	10.389050	10.388900	3.39e-01	5.67e-06	1.56e-04	5.18e-06

Рис. 16: Результаты решения задачи Коши

4.2 Задание 4.2

Постановка задачи: Реализовать метод стрельбы и конечно-разностный метод решения краевой задачи для ОДУ в виде программ. С использованием разработанного программного обеспечения решить краевую задачу для обыкновенного дифференциального уравнения 2-го порядка на указанном отрезке. Оценить погрешность численного решения с использованием метода Рунге – Ромберга и путем сравнения с точным решением.

17	$(x^2-1)y''+(x-3)y'-y=0,$ $y'(0)=0,$ $y'(1)+y(1)=-0.75$	$y(x) = x - 3 + \frac{1}{x+1}$
----	---	--------------------------------

4.2.1 Главные части кода

```
1 import math
2
3 # --- 1. Исходные данные Варианта 17 ---
4 x0, x_end = 0.0, 1.0
5 h = 0.1
6
7
8 def exact_sol(x):
9     return x - 3 + 1.0 / (x + 1)
10
11 def p_func(x):
12     denom = x ** 2 - 1.0
13     if abs(denom) < 1e-15:
14         denom = -1e-15 if denom <= 0 else 1e-15
15     return (x - 3.0) / denom
16
17
18 def q_func(x):
19     denom = x ** 2 - 1.0
20     if abs(denom) < 1e-15:
21         denom = -1e-15 if denom <= 0 else 1e-15
22     return -1.0 / denom
23
24
25 def f_func(x):
26     return 0.0
27
28
29 def f_z(x, y, z):
30     return f_func(x) - p_func(x) * z - q_func(x) * y
31
32 def rk4_step(x, y, z, h_step):
33     k1 = h_step * z
34     l1 = h_step * f_z(x, y, z)
35
36     k2 = h_step * (z + 0.5 * l1)
37     l2 = h_step * f_z(x + 0.5 * h_step, y + 0.5 * k1, z + 0.5 * l1)
38
39     k3 = h_step * (z + 0.5 * l2)
40     l3 = h_step * f_z(x + 0.5 * h_step, y + 0.5 * k2, z + 0.5 * l2)
41
42     k4 = h_step * (z + l3)
```

```

43     l4 = h_step * f_z(x + h_step, y + k3, z + l3)
44
45     return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0, z + (l1 + 2 * l2 + 2 *
46     l3 + l4) / 6.0
47
48 def solve_ivp_rk4(y_start, z_start, h_step):
49     N_steps = int(round((x_end - x0) / h_step))
50     x_arr = [x0 + i * h_step for i in range(N_steps + 1)]
51     y_arr = [0.0] * (N_steps + 1)
52     z_arr = [0.0] * (N_steps + 1)
53
54     y_arr[0], z_arr[0] = y_start, z_start
55     for k in range(N_steps):
56         y_arr[k + 1], z_arr[k + 1] = rk4_step(x_arr[k], y_arr[k], z_arr[k],
57         h_step)
58     return x_arr, y_arr, z_arr
59
60 def shooting_method(h_step):
61     def phi(eta):
62         _, y_res, z_res = solve_ivp_rk4(eta, 0.0, h_step)
63         return z_res[-1] + y_res[-1] + 0.75
64
65     eta0, eta1 = -2.0, -1.0
66     phi0, phi1 = phi(eta0), phi(eta1)
67     eps = 1e-7
68
69     while abs(phi1) > eps:
70         if abs(phi1 - phi0) < 1e-12:
71             break
72         eta_next = eta1 - (eta1 - eta0) / (phi1 - phi0) * phi1
73         eta0, eta1 = eta1, eta_next
74         phi0, phi1 = phi1, phi(eta1)
75
76     x_arr, y_arr, _ = solve_ivp_rk4(eta1, 0.0, h_step)
77     return x_arr, y_arr
78
79
80 def finite_difference_method(h_step):
81     N = int(round((x_end - x0) / h_step))
82     x_arr = [x0 + i * h_step for i in range(N + 1)]
83
84     A = [0.0] * (N + 1)
85     B = [0.0] * (N + 1)
86     C = [0.0] * (N + 1)
87     D = [0.0] * (N + 1)
88
89     C[0] = -1.0
90     B[0] = 1.0
91     D[0] = 0.0
92
93     for k in range(1, N):
94         pk = p_func(x_arr[k])
95         qk = q_func(x_arr[k])
96         fk = f_func(x_arr[k])
97
98         A[k] = 1.0 - (pk * h_step) / 2.0
99         C[k] = -2.0 + (qk * h_step ** 2)
100        B[k] = 1.0 + (pk * h_step) / 2.0

```

```

101     D[k] = fk * h_step ** 2
102
103     A[N] = -1.0
104     C[N] = 1.0 + h_step
105     D[N] = -0.75 * h_step
106
107     alpha = [0.0] * (N + 1)
108     beta = [0.0] * (N + 1)
109
110     alpha[1] = -B[0] / C[0]
111     beta[1] = D[0] / C[0]
112
113     for k in range(1, N):
114         denom = C[k] + A[k] * alpha[k]
115         alpha[k + 1] = -B[k] / denom
116         beta[k + 1] = (D[k] - A[k] * beta[k]) / denom
117
118     y_arr = [0.0] * (N + 1)
119     y_arr[N] = (D[N] - A[N] * beta[N]) / (C[N] + A[N] * alpha[N])
120
121     for k in range(N - 1, -1, -1):
122         y_arr[k] = alpha[k + 1] * y_arr[k + 1] + beta[k + 1]
123
124     return x_arr, y_arr

```

Листинг 14: Метод стрельбы и КРМ

4.2.2 Результаты работы программы

=== РЕЗУЛЬТАТЫ РЕШЕНИЯ КРАЕВОЙ ЗАДАЧИ (ВАРИАНТ 17) ===							
x	Exact	Shooting	FDM	Err Shoot	Err FDM	R-R Shoot	R-R FDM

0.0	-2.000000	-0.000000	-1.930651	2.00e+00	6.93e-02	1.77e-12	7.31e-02
0.1	-1.990909	-0.000000	-1.930651	1.99e+00	6.03e-02	-	-
0.2	-1.966667	-0.000000	-1.913641	1.97e+00	5.30e-02	1.74e-12	5.61e-02
0.3	-1.930769	-0.000000	-1.883564	1.93e+00	4.72e-02	-	-
0.4	-1.885714	-0.000000	-1.843233	1.89e+00	4.25e-02	1.67e-12	4.56e-02
0.5	-1.833333	-0.000000	-1.794711	1.83e+00	3.86e-02	-	-
0.6	-1.775000	-0.000000	-1.739541	1.77e+00	3.55e-02	1.57e-12	3.89e-02
0.7	-1.711765	-0.000000	-1.678904	1.71e+00	3.29e-02	-	-
0.8	-1.644444	-0.000000	-1.613719	1.64e+00	3.07e-02	1.46e-12	3.44e-02
0.9	-1.573684	-0.000000	-1.544712	1.57e+00	2.90e-02	-	-
1.0	-1.500000	-0.000000	-1.472466	1.50e+00	2.75e-02	1.33e-12	3.14e-02

Рис. 17: Результаты решения краевой задачи